15,471,903 members Sign in

articles Q&A forums stuff lounge ? Search for articles, questions,

A tool to order the window buttons in your taskbar



Nicholas Butler
9 Aug 2005 CPOL

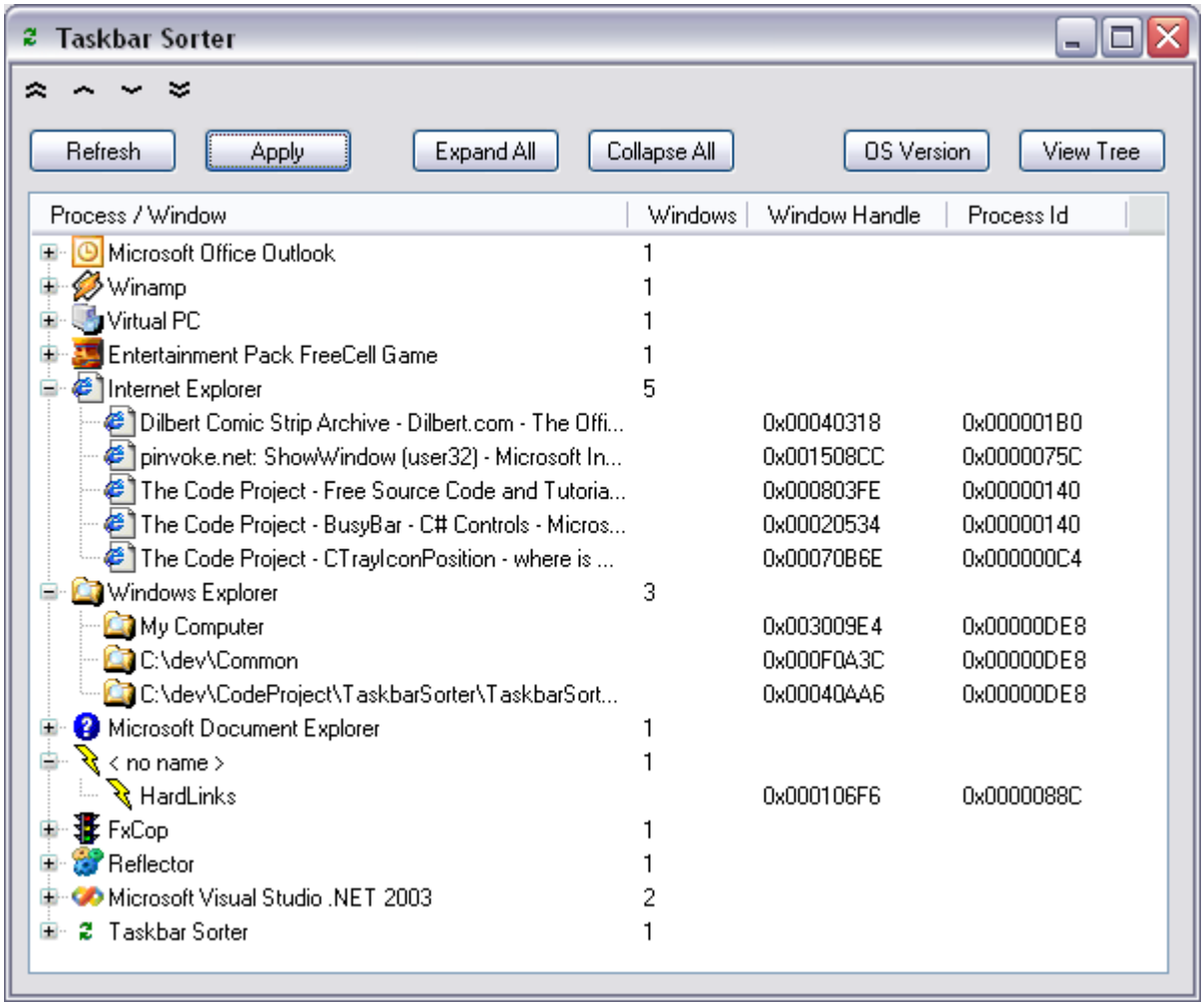
Rate me: 4.90/5 (53 votes)

Move your window buttons into your preferred order. WinXP or later only.

Download application - 86.6 Kb

Download source - 94.4 Kb

Note: This tool only works on Windows XP and later!



Introduction

This tool allows you to alter the order of the window buttons in your taskbar.

Background

I normally have about 20 windows open, and switching between them is a lot quicker if they are in my "normal" order. For instance, I always like Outlook to be the first button in my taskbar, so I know where to click when I get mail. The problem is that when Outlook hangs, I have to kill its process. Then when I restart it, it appears at the end of the list, and is soon hidden among my other windows. With this tool, I can easily move Outlook back to the start where it belongs, without closing and re-opening all my other windows.

This problem is a pet peeve of mine - if the order of your windows doesn't bother you, then you don't need this tool :)

How it works

This tool uses a lot of PInvoke calls - if you want to see the details, look in the "*Common/Kernel32.cs*" and "*Common/User32.cs*" files in the source. It also uses an `unsafe` method (my first one), which made me feel a bit dirty :). I started to write it in C++, but after four years of C#, it was just too painful.

The basic idea is to get a list of the windows in the taskbar, show them and allow them to be reordered, and then apply the new ordering. Sounds easy :)

Getting a list of the windows

This proved to be the hardest part. Fortunately, I found a way to do it, but it only works on Windows XP and later. Windows 2000 uses a different window hierarchy, and I don't know (or care :) about earlier OSs.

Getting the buttons window handle

Using Spy++, I found that the buttons window in the taskbar is actually a `ToolbarWindow32` - one of the common Windows controls. This was luck, as Microsoft could easily have used a custom window class. As it is, we know quite a lot about `ToolbarWindow32`. The first thing to do is get its window handle. We can do this using `GetDesktopWindow` and `FindWindowEx`:

C#

```
IntPtr hDesktop = User32.GetDesktopWindow();
IntPtr hTray    = User32.FindWindowEx( hDesktop , 0, "Shell_TrayWnd"    , null );
IntPtr hReBar   = User32.FindWindowEx( hTray    , 0, "ReBarWindow32"    , null );
IntPtr hTask    = User32.FindWindowEx( hReBar   , 0, "MSTaskSwWClass"   , null );
IntPtr hToolbar = User32.FindWindowEx( hTask    , 0, "ToolbarWindow32" , null );
```

This just walks down the window hierarchy from the desktop window. We now have a handle on the `ToolbarWindow32`, and can start the fun stuff :)

Getting the number of buttons

This is easy, as we know about the `TB_BUTTONCOUNT` message. All we have to do is send this message using our window handle, and it will return the count:

C#

```
UInt32 count = User32.SendMessage( _ToolbarWindowHandle, TB.BUTTONCOUNT, 0, 0 );
```

Note that the window count is more than we would expect. See Groups for an explanation.

Getting each button's info

This also should be easy, because we know about the `TB_GETBUTTON` message. It's a bit more complicated because we are accessing a `ToolbarWindow32` from a different process, so we have to do

a bit of memory management. See Cross-process memory access for details. It also uses an `unsafe` block, as we need to pass the addresses of structures as pointers. I will concentrate on the algorithm for now:

```
TBBUTTON tbButton;
TBBUTTON* ipRemoteBuffer = & tbButton; // unsafe
for ( int i = 0 ; i < count ; i++ )
{
    User32.SendMessage(
        hToolbar,
        TB.GETBUTTON,
        ( IntPtr ) i,
        ipRemoteBuffer );
}
```

Note that this is a pseudo-code - it doesn't compile, it just gives you an idea of the algorithm.

We now have a `TBBUTTON` structure filled with information for each button. If we look at the `fsState` field, we find that some of the buttons are hidden, and that there are the expected number of visible buttons. Lucky again :) Note that the `TBBUTTON` structure has a 32-bit `dwData` field that is for user data. This will come in handy later on...

Getting the button text

The `TBBUTTON` structure has a `String` field, but it's not that easy to use. It's easier to use a `TB_GETBUTTONTEXT` message to let the control do the work:

C#

```
int chars = ( int ) User32.SendMessage(
    hToolbar,
    TB.GETBUTTONTEXTW,
    ( IntPtr ) tbButton.idCommand,
    ipRemoteBuffer );
```

Note that this message takes a `CommandId`, which we got from the `TBBUTTON` structure.

Getting the window handle

This was the really lucky part. I thought to myself: "Where would I keep the window handle?". They must keep it somewhere, to enable the activation of the correct window when a button is selected. The obvious place to keep it would be in a structure for each button, and the obvious place to keep the pointer to this structure would be in the `dwData` field of each `TBBUTTON`.

So I had a look at the `dwData` fields, and they appeared to be pointers. OK so far. Then I had a look at the memory they pointed to, and there they were: the first field stores the window handle :)))

Microsoft developers aren't so different, after all :)

Using the window handle

The rest is easy. I used Thomas Caudal [^] 's TreeListView [^] which made the UI easy. A bit of glue to handle moving the items around, and then the "Apply" button.

All we have to do to apply the new order is to hide all the windows using `ShowWindow`, and then show them one at a time in the desired order. Easy peasy :)

Points of interest

A few things could do with a bit more explanation.

Groups

In XP and later, you have the option of grouping similar taskbar buttons together. This tool works with this switched on or off. In either case, Explorer adds a dummy button, which is hidden, at the start of each group. This is why `TB_BUTTONCOUNT` reports more buttons than are visible. These extra buttons are easy to identify, as they don't have a window handle.

Out of interest, the behaviour is controlled by a couple of registry entries:

- "*HKCU\Software\Microsoft\Windows\CurrentVersion \Explorer\Advanced\TaskbarGlomming*" is 0 if grouping is switched off, and 1 if it is on.
- "*HKCU\Software\Microsoft\Windows\CurrentVersion \Explorer\Advanced\TaskbarGroupSize*" is the minimum number of windows before they are collapsed into one button. I have this set to 99 to effectively turn it off. You can set this value with TweakUI from the XP PowerToys (it's probably safer).

Cross-process memory access

This was a tricky one. Some of the required Win32 functions use structures to move information around. This works well if you and your target are in the same process space, but falls apart in our case. The naive implementation doesn't work. If you declare a structure, and get a pointer to it to pass into your function, the pointer is to some address in your virtual memory space. The function can't know this, and thinks that it is pointing to a block of its virtual memory. This can only lead to General Protection Faults at best.

The solution is to allocate space for your structure in Explorer's virtual memory, call the function, and then copy the data back into your virtual memory so you can use it. This works without elevated privileges because Explorer runs under the local user account. As we use a few functions one after the other, I just allocate a page of memory, and use this for all the structures.

The basic algorithm is this:

- use `GetWindowThreadProcessId` to get the process ID from a window handle.
- use `OpenProcess` to turn the process ID into a process handle.
- use `VirtualAllocEx` to allocate some memory in Explorer's virtual memory space.
- call the required API function to fill the buffer.
- use `ReadProcessMemory` to copy the buffer into our virtual memory space.
- don't forget to use `VirtualFreeEx` to free the buffer, and `CloseHandle` to finish.

I will just show one call here, the others are similar:

C# Shrink ▲

```
private unsafe bool GetTBButton(
    IntPtr hToolbar,
    int i, // button id
    ref TBBUTTON tbButton,
    ... )
{
    // One page
    const int BUFFER_SIZE = 0x1000;

    byte[] localBuffer = new byte[ BUFFER_SIZE ];

    UInt32 processId = 0;
    UInt32 threadId =
        User32.GetWindowThreadProcessId(
            hToolbar,
            out processId );

    IntPtr hProcess =
        Kernel32.OpenProcess(
            ProcessRights.ALL_ACCESS,
            false,
            processId );

    if ( hProcess == IntPtr.Zero ) return false;

    IntPtr ipRemoteBuffer = Kernel32.VirtualAllocEx(
        hProcess,
        IntPtr.Zero,
        new UIntPtr( BUFFER_SIZE ),
        MemAllocationType.COMMIT,
        MemoryProtection.PAGE_READWRITE );

    if ( ipRemoteBuffer == IntPtr.Zero ) return false;

    // TBButton
    fixed ( TBBUTTON* pTBButton = & tbButton )
    {
        IntPtr ipTBButton = new IntPtr( pTBButton );

        int b = ( int ) User32.SendMessage(
            hToolbar,
            TB.GETBUTTON,
```

```
            ( IntPtr ) i,
            ipRemoteBuffer );

        if ( b == 0 ) { Debug.Assert( false ); return false; }

        Int32 dwBytesRead = 0;
        IntPtr ipBytesRead = new IntPtr( & dwBytesRead );

        bool b2 = Kernel32.ReadProcessMemory(
            hProcess,
            ipRemoteBuffer,
            ipTBButton,
            new UIntPtr( ( uint ) sizeof( TBBUTTON ) ),
            ipBytesRead );

        if ( ! b2 ) { Debug.Assert( false ); return false; }
    }

    ...

    Kernel32.VirtualFreeEx(
        hProcess,
        ipRemoteBuffer,
        UIntPtr.Zero,
        MemAllocationType.RELEASE );

    Kernel32.CloseHandle( hProcess );

    return true;
}
```

The `unsafe` keyword allows us to use pointers, and the `fixed` keyword locks the object in memory, so that the GC doesn't move it while we copy the data back to our virtual memory space. Apart from that, it's just a bit of P/Invoke. Check out the "*Common\Kernel32.cs*" and "*Common\User32.cs*" files in the source if you're interested.

Icons

This is just a bit of fluff. The `TreeListView` doesn't work properly without an `ImageList`, so I added one. Then I thought it would be good to show the default window icons to distinguish the items. My first attempt wasn't too successful. I used `GetClassLong` with the `GCL_HICONSM` parameter to get the small icon associated with the window class. This works for most applications, but not for my .NET apps. It seems that the framework generates a new window class each time an application runs, but it doesn't set the icon handle - nice :) These apps do, however, correctly return an icon handle when sent a `WM_GETICON` message, so that's what I did in the end. I also set the window class icon handle for this tool in the main form's `OnHandleCreated` override, but this isn't necessary.

Shameless plugs

I used the code from a couple of my previous articles in this project. I used my OSVersion [^] class to check that we're running on XP or later. And I used my TreeCollection [^] as a backing store. I also

used the `GlobalMemoryStatusEx` API to display memory usage in the "OS Version" dialog - just for fun :)

Conclusion

Well, that's about it. If you just want to use this tool, that's fine - I hope you find it useful. If you want to dig a little deeper, then you can look at the code as an example of P/Invoke and cross-process memory access.

I just want to mention the pinvoke.net [^] wiki site - very handy and mostly accurate :)

History

- 8th August 2005 - Version 2.
  - `hIcon` bug fixed.
  - changed icon.
- 27th May 2005 - Version 1.

License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)

Written By

Nicholas Butler

United Kingdom

I discovered C# and .NET 1.0 Beta 1 in late 2000 and loved them immediately.
I have been writing software professionally in C# ever since

In real life, I have spent 3 years travelling abroad,
I have held a UK Private Pilots Licence for 20 years,
and I am a PADI Divemaster.

I now live near idyllic Bournemouth in England.

I can work 'virtually' anywhere!

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

It does not work in Windows 10
Win32nipuh 8-May-16 21:22

Unknown OS
gultron 19-May-15 5:56

I have this message when opening TaskbarSorter.exe

Unknown OS

Could not determine Operating System

Error:
Invalid Win32NT MajorVersion: 6.

I am using Windows Enterprise.

Please response to alexandreviau AT gmail DOT com

Sign In · View Thread

WARNING!!!
Matthew Miller 16-Oct-14 18:41

Re: WARNING!!!
Ray K 25-Feb-15 21:51

Vista x64 not supported
zpro 14-Mar-13 2:32

adding textbox on top of the task bar
princtreme1 10-Sep-12 1:46

Excellent
Knasenmc3 14-Jan-11 3:57

My vote of 5
chatravin 3-Oct-10 8:24

Windows 7 Problems
DrWatsonCoding 25-Aug-10 8:31

Nice but...
gbrlhtclcq 25-Aug-09 23:01

Change button Text
C4702 14-Dec-08 5:53

Thanks!!!
Member 603855 22-Aug-08 12:25

Hide - Unhide aswell?
Stumproot 25-Feb-08 0:03

Re: Hide - Unhide aswell?
Member 603855 22-Aug-08 12:24

Inserting button to windows explorer from a shell extension.
Shafeeque O.K. 26-Dec-07 3:21

The answer to my dreams!
susan hopkin 4-Dec-07 17:04

Does not work under Windows Vista x64
Spaceman28 8-Nov-07 5:51

Re: Does not work under Windows Vista x64
Member 603855 22-Aug-08 12:27

Re: Does not work under Windows Vista x64
Decad 10-Oct-08 14:20

Re: Does not work under Windows Vista x64
Elvandar 6-Apr-09 23:20

no need for GetDesktopWindow()
goher 9-May-07 0:01

Not supported for C++

coka2cora 9-Jan-07 22:26

Alternate method that is cleaner

Bob_Friday 7-Dec-06 14:05

Support for Vista

Mark Stafford 23-Sep-06 9:11

Can I get new added or removed button event?

jucysoft 14-Feb-06 17:26

Refresh

1 2 3 Next ▷

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2005 by Nicholas Butler
Everything else Copyright © CodeProject, 1999-2022

Web03 2.8:2022-10-01:1